

COS214 Practical 3

Student Numbers:

- u25138309 – Christian Nunes
- u25433726 – Yoshua SMit
- u25108582 – Jonathan Witter

Task 1

1.1)

The event theme is Esports Tournament. This is an organised event for competitive video games. There are 2 teams (consisting of 5 players) which will compete against each other.

Grouping/areas are:

- Main Venue (where everything is happening – Like a large hall)
- Stage (Where the teams actually sit and compete)
- Lounge (Area to relax, buy food and merch)

Event Units are:

- Table/Set (Where the team will have their pc setups)
- RefTeam (The referees overseeing the games)
- Benches (Where spectators can sit and watch the games)
- Food Stall (People can buy food/drinks)
- Merch Stall (Game/tournament related items that people can buy)

1.2)

Composite Pattern

Component – EventComponent

Leaf – EventUnit

Composite – EventGroup

Concrete Leaves – SpectatorBench, TeamSet, RefTeam, FoodStall, MerchStall

Concrete Composites – MainVenue, Stage, Lounge

Observer Pattern

Observer – Observer

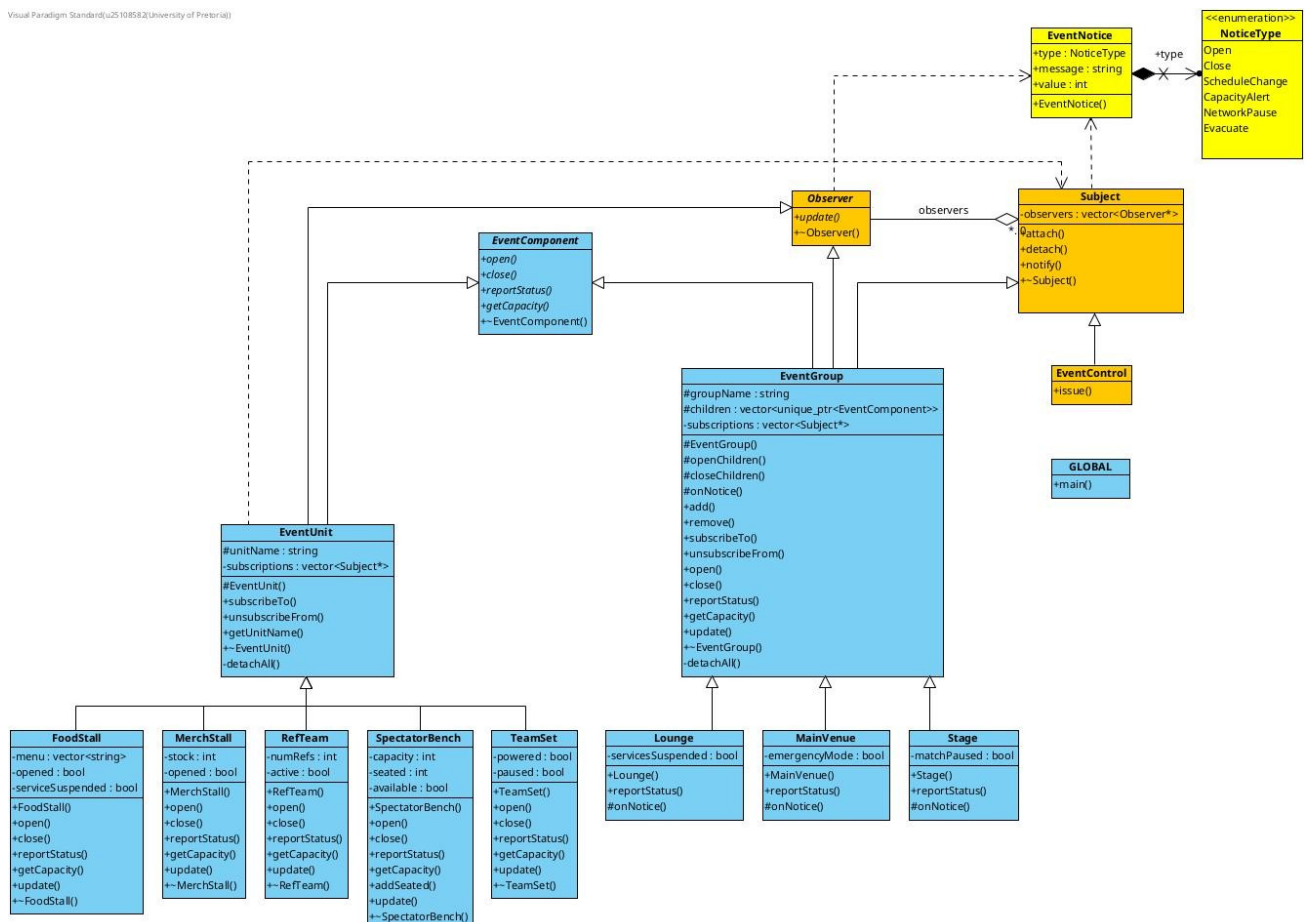
Subject – Subject

ConcreteSubject – EventControl

ConcreteObserver – EventGroup

1.3)

Visual Paradigm Standard (v25.10.58.2) (University of Pretoria)



1.4) a) The EventFlow structure is a genuine Composite part-whole tree because each EventGroup owns its child EventComponent objects using `unique_ptr<EventComponent>`. A MainVenue owns areas such as a Stage and Lounge, and those areas in turn own their operational units, such as TeamSet, FoodStall, MerchStall, RefTeam, and SpectatorBench objects. A component can therefore have one owning parent, while a group can contain both individual leaves and further groups.

Both EventGroup objects and EventUnit objects implement the EventComponent interface. This lets a client call `open()`, `close()`, `reportStatus()`, or `getCapacity()` without knowing whether it is operating on one unit or a whole subtree. EventGroup implements these operations by delegating recursively to its owned children, while each leaf provides its own specific response.

b) Observer is required because EventControl must inform a changing set of event areas and units about notices without containing hard coded calls to every concrete class. Direct calls from EventControl to Stage, Lounge, TeamSet, FoodStall, and other concrete types would tightly couple the control centre to the complete event layout. Adding, moving, or removing a unit would then require modifying EventControl (which also violates the open closed principle)

Thus, we made EventControl a Subject that sends notices to its registered Observer objects. An EventGroup can receive a notice from its parent and rebroadcast it to the observers registered below it. This produces a cascade through the venue, its areas, and their units while preserving the different responses implemented by each concrete class. For example, a network pause can pause a TeamSet and a Stage, while an evacuation can close food and merchandise stalls but keep the RefTeam active to direct people safely.

c) The Composite tree owns event components. EventGroup stores its children in a `vector<unique_ptr<EventComponent>>`, so adding a child transfers sole ownership to the group. When a group is destroyed, its `unique_ptr` objects destroy the complete owned subtree exactly once. A component is moved between groups by calling `remove()`, which returns its `unique_ptr` to the caller, then passing that `unique_ptr` to the receiving group's `add()` method.

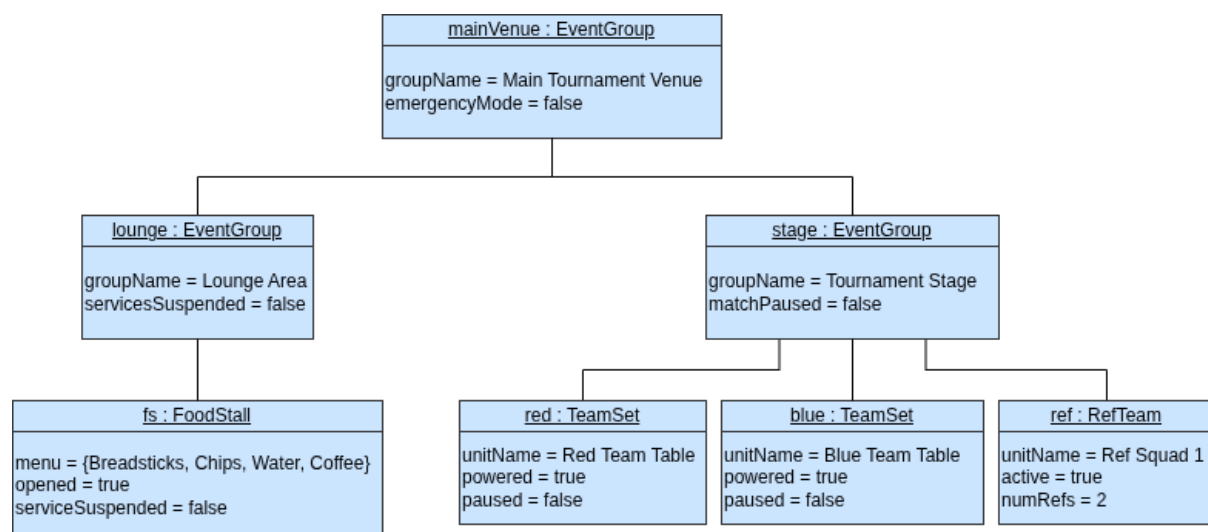
d) Subjects do not own observers. Subject stores non-owning Observer* pointers in its registration list because an observer can be registered with more than one subject and because observer registration is a communication relationship, not a lifetime relationship. Subject::attach() ignores duplicate registrations and Subject::detach() safely removes an observer if it is present.

Each EventUnit and EventGroup tracks the Subject objects to which it has subscribed. Its destructor calls detachAll(), removing itself from those subjects before the object is destroyed. This prevents Subject registration lists from retaining dangling Observer* pointers.

e) EventGroup has two separate responsibilities in two separate collaborations. As an Observer, it receives an EventNotice from a parent-level subject, such as EventControl or the MainVenue. As a Subject, it notifies its own registered child observers after it has received that notice. These roles are not opposites.

The first role describes how a group hears about an event-wide change, while the second describes how it propagates a relevant change within its contained area. This is what enables a notice to cascade through multiple runtime levels without EventControl knowing every leaf object.

Task 2



2.4)

Task 3

3.5) We decided to go with the push model. Our notify function passes the notice directly to update. The exact information transferred is one EventNotice value containing:

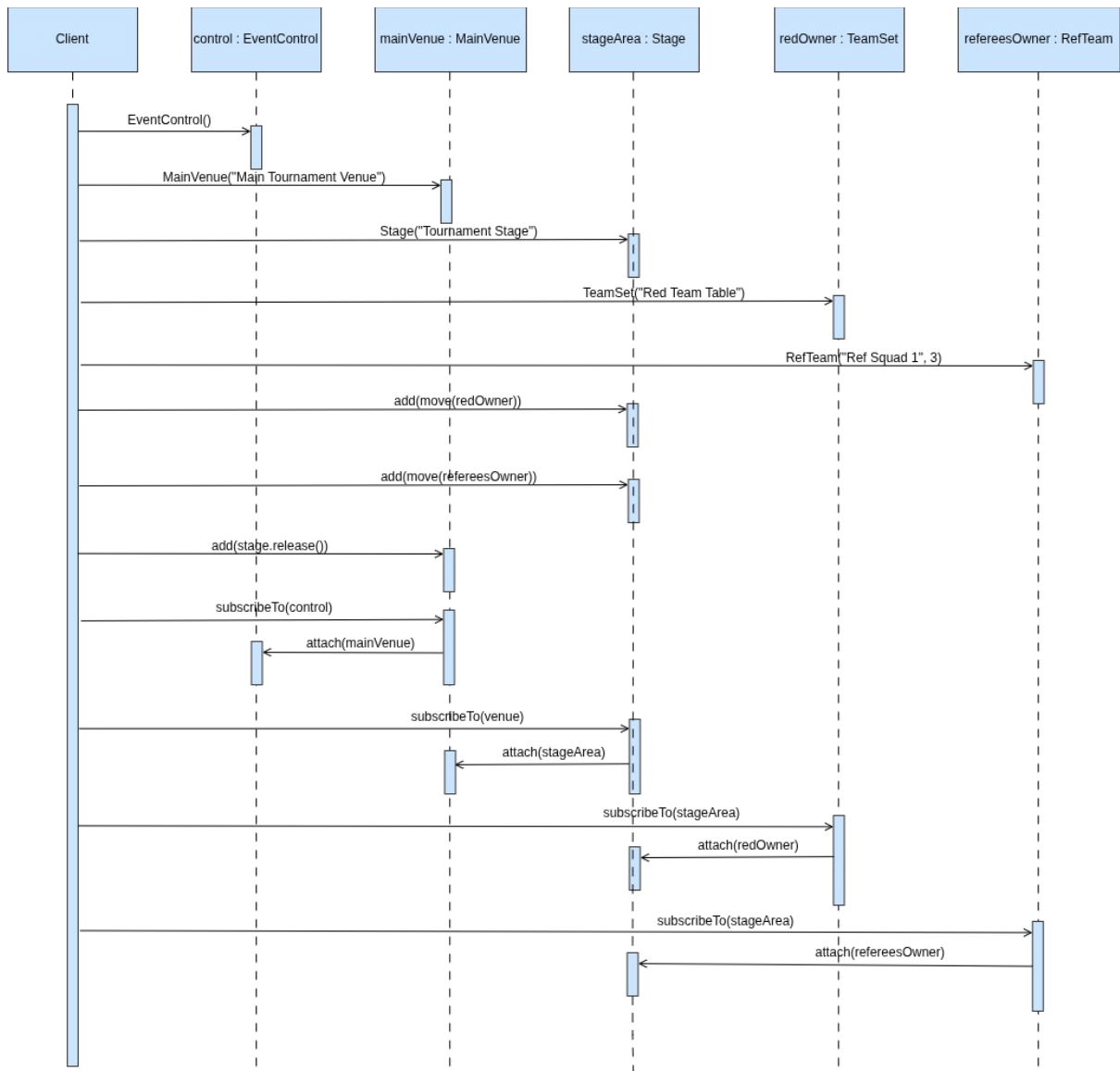
- type: a NoticeType enum value. (e.g. Open, Close, Evacuate etc.)
- message: a string that explains the notice, (e.g. "Tournament network instability".)
- value: an int carrying an optional numeric value, (e.g. the capacity threshold used by a CapacityAlert.)

The main advantage of push is that observers immediately receive the precise information needed to react. They don't need to query a Subject for current state during an urgent notice. It also keeps the notification cascade simple. Each EventGroup can forward the same EventNotice to its local observers.

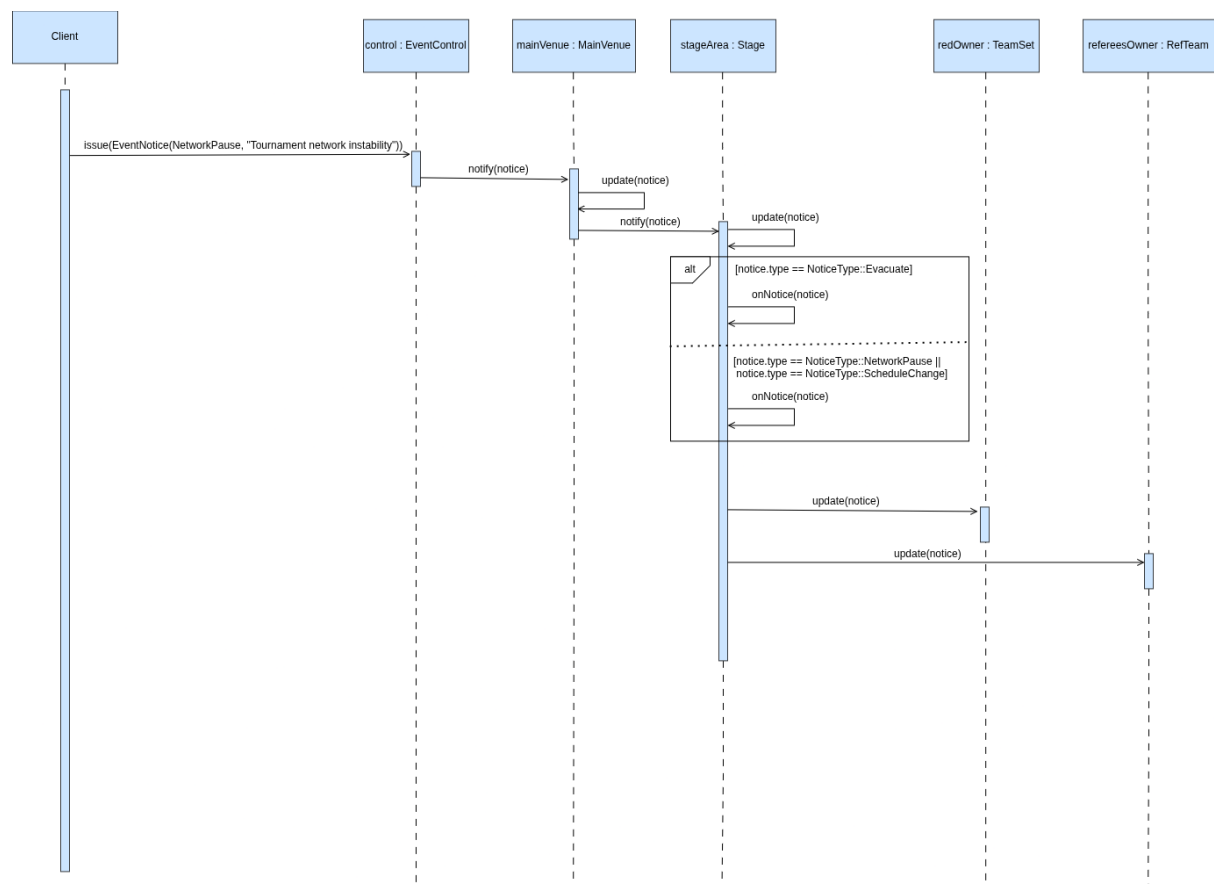
The trade-off is that EventNotice had to be designed to carry enough information for all relevant observers. If a notice later needs more context, the value type must be extended and observers may need updates. A pull model would keep the update message smaller and let each observer query only the state it needs, but would couple observers to Subject query operations and add extra calls at notification time.

Task 5

5.1)



5.3)



Task 7

7.4) Using Git and GitHub, we collaborated on the development of this project, making full use of branching, merging and version control. We created a dev branch which enabled the use of rapid development, allowing us to experiment and figure out what we wanted and needed to develop. We categorized our contributions with the commit system in a way that was descriptive and concise and as we completed a feature, we merged the dev branch into the main branch, ready for the final submission when it was ready.